How to compile Bash script into binary

A Bash script is basically a text file containing commands written in Bash command language. The script below, for example, will greet the name that we supply as a parameter:

hello.sh

```
#!/bin/bash
echo Hello, $1!
```


You can execute the shell script like the following:

```
$ bash hello.sh Alice
Hello, Alice!
```

File type details of the script can be viewed using the file command:

```
$ file hello.sh
hello.sh: Bourne-Again shell script, ASCII text executable
```

One of the caveats of using such script is that the code is clearly visible, and it might not be as performant as running a compiled program.

shc is a Bash script compiler. You can use it to hide the source code (as some would say "encrypt") of the Bash script and remove dependency to Bash interpreter by compiling your Bash script into an executable binary from the command line.

Steps to compile Bash script into binary:

1. Install shc and required libraries.

```
$ sudo apt update && sudo apt install --assume-yes gcc shc  #For //U
##### snipped
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu cpp cpp-8 gcc-8
  libbinutils libc-dev-bin libc6-dev libcc1-0 libgcc-8-dev libgomp1
  liblsan0 libmpc3 libmpx2 libquadmath0 libtsan0 libubsan1 linux-lib
Suggested packages:
  binutils-doc cpp-doc gcc-8-locales gcc-multilib make autoconf autor
  gdb gcc-doc gcc-8-multilib gcc-8-doc libgcc1-dbg libgomp1-dbg libi
  libasan5-dbg liblsan0-dbg libtsan0-dbg libubsan1-dbg libmpx2-dbg l
  glibc-doc
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu cpp cpp-8 gcc g
  libbinutils libc-dev-bin libc6-dev libcc1-0 libgcc-8-dev libgomp1
  liblsan0 libmpc3 libmpx2 libquadmath0 libtsan0 libubsan1 linux-lib
0 upgraded, 26 newly installed, 0 to remove and 0 not upgraded.
Need to get 27.8 MB of archives.
##### snipped
```

> For other Linux distributions.
>
> - Install gcc and other basic development tools.
> - Download from and refer to https://github.com/neurobin/shc

2. Compile your script using shc.

```
$ shc -f hello.sh
```

3. Check generated files.

```
$ ls -l hello*
-rw-rw-r-- 1 user user    29 Mar 14 07:37 hello.sh
-rwxrwxr-x 1 user user 14960 Mar 14 07:39 hello.sh.x
-rw-rw-r-- 1 user user 10047 Mar 14 07:39 hello.sh.x.c
```

.sh is the original script.
sh.x is the compiled binary.
.sh.x.c is the C source code generated from the .sh file prior to compiling to .sh.x.

hello.sh.x permission is automatically set as executable

4. Check file type (optional).

```
$ file hello.sh.x
hello.sh.x: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),
```

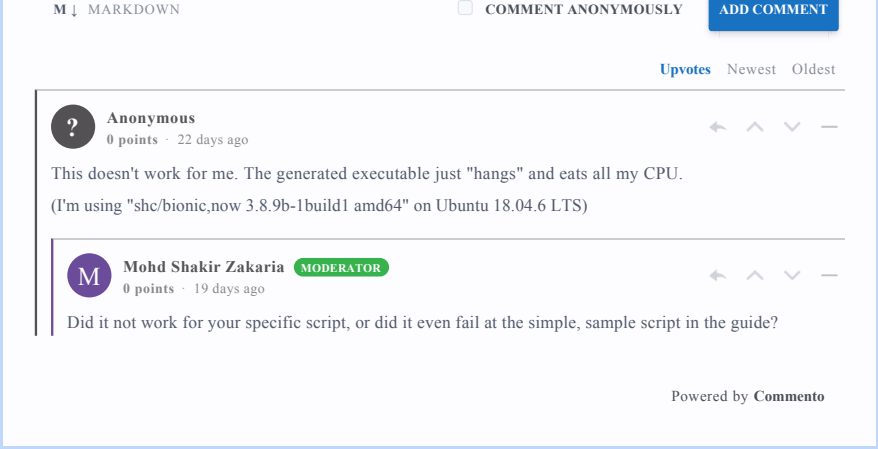
5. Rename executable (optional).

```
$ mv hello.sh.x hello
```

6. Check file execution. (optional).

```
$ ./hello Alice
Hello, Alice!
```

Author: **Mohd Shakir Zakaria**
Cloud architect by profession but always consider himself as a developer, entrepreneur and an opensource enthusiast.


M ↓ MARKDOWN
COMMENT ANONYMOUSLY
ADD COMMENT
Upvotes Newest Oldest
Anonymous
0 points · 22 days ago
This doesn't work for me. The generated executable just "hangs" and eats all my CPU.
(I'm using "shc/bionic,now 3.8.9b-1build1 amd64" on Ubuntu 18.04.6 LTS)
Mohd Shakir Zakaria MODERATOR
0 points · 19 days ago
Did it not work for your specific script, or did it even fail at the simple, sample script in the guide?
Powered by Commento

Back to top